

self vs Self in Rust

The receiver and the type — a practical reference with worked examples

The whole rule in one line

`self` is the **value** (the instance you called the method on). `Self` is the **type** (whatever the enclosing `impl` or `trait` block is about). Lowercase is data; capitalised is a type name.

Throughout this document code is coloured so the distinction is visible at a glance: the value `self` is blue, the type `Self` is purple. Every example here was compiled and run with `rustc 1.98.1` (edition 2024); the appendix lists the complete program and its real output.

1. self — the value

`self` is not a keyword you sprinkle anywhere: it is meaningful as the **first parameter of a function inside an `impl` block**, where it names the receiver. The three familiar spellings are pure sugar for an ordinary typed parameter — and notice that the type is written with `Self`:

Sugar	What it really means	Effect on the caller
<code>&self</code>	<code>self: &Self</code>	shared borrow — caller keeps the value
<code>&mut self</code>	<code>self: &mut Self</code>	exclusive borrow — caller keeps the value
<code>self</code>	<code>self: Self</code>	moves the value in — caller loses it
<code>mut self</code>	<code>mut self: Self</code>	moves it in, callee may reassign fields
(none)	—	associated function: no instance at all

You can write the long form yourself, which proves the sugar is nothing special:

Desugaring, spelled out

```
struct T(i32);

impl T {
    fn a(self: &Self) -> i32 { self.0 }           // identical to &self
    fn b(self: &mut Self) { self.0 += 1 }         // identical to &mut self
    fn c(self: Self) -> i32 { self.0 }             // identical to self
    fn d(self: Box<Self>) -> i32 { self.0 * 2 }    // an "arbitrary self type"
}
```

A method call is then just a function call with the receiver passed in: `t.a()` is `T::a(&t)`, and `t.b()` is `T::b(&mut t)`. Keeping that rewrite in mind explains almost every borrow error you will meet.

2. &self — shared borrow (the default choice)

Use `&self` whenever the method only **reads**. The caller keeps ownership, and any number of `&self` borrows may be alive simultaneously.

```

struct Person { name: String, age: u32 }

impl Person {
    // 1. format / report
    fn describe(&self) -> String {
        format!("{}", ({}), self.name, self.age)
    }

    // 2. a predicate over the fields
    fn is_adult(&self) -> bool { self.age >= 18 }

    // 3. hand out a reference borrowed *from* self
    fn name(&self) -> &str { &self.name }

    // 4. two shared borrows at the same time: fine
    fn same_age(&self, other: &Self) -> bool { self.age == other.age }
}

let a = Person { name: "Bora".into(), age: 32 };
let b = Person { name: "Ada".into(), age: 32 };
println!("{}", a.describe(), a.is_adult(), a.name(), a.same_age(&b));
println!("{}", a.describe()); // `a` is untouched and still usable

```

Note example 3: returning `&str` from `&self` is shorthand for `fn name<'x>(&'x self) -> &'x str` — the returned reference is tied to the borrow of `self`, so the compiler will not let the `Person` die while the `&str` lives.

This is the fix for the most common beginner move-error. A method declared `fn print(self)` consumes the receiver, so `p.print()` moves `p` even if you write `&p.print()` — that parses as `&(p.print())`, which calls the method first and then borrows the returned `String`. Change the signature to `&self` and the problem disappears.

```

// before: p is moved away by the call
impl Person { fn print(self) -> String { /* ... */ String::new() } }

// after: p is only borrowed, and stays usable
impl Person { fn print(&self) -> String { /* ... */ String::new() } }

```

3. `&mut self` — exclusive borrow

Use `&mut self` when the method **changes** the value. Ownership still stays with the caller; what changes is exclusivity: while a `&mut` borrow is alive, no other borrow of that value may exist, not even a shared one. That guarantee is what makes in-place mutation safe.

```

#[derive(Debug)]
struct Stack { items: Vec<i32>, pushes: usize }

impl Stack {
    fn new() -> Self { Self { items: Vec::new(), pushes: 0 } }

    // 1. plain mutation of fields
    fn push(&mut self, v: i32) {
        self.items.push(v);
        self.pushes += 1;
    }

    // 2. mutate, and return owned data taken out of self
    fn pop(&mut self) -> Option<i32> { self.items.pop() }

    // 3. hand out a mutable reference *into* self
    fn top_mut(&mut self) -> Option<&mut i32> { self.items.last_mut() }

    // 4. return &mut Self to allow chaining
    fn with(&mut self, v: i32) -> &mut Self { self.push(v); self }

    // 5. reset in place
    fn clear(&mut self) { self.items.clear(); }

    fn len(&self) -> usize { self.items.len() } // reads: &self
}

```

Calling them — with the real printed output

```

let mut s = Stack::new(); // `mut` on the binding is required
s.push(1);
s.push(2);

if let Some(t) = s.top_mut() {
    *t *= 100; // write through the &mut i32
} // ...borrow ends here

println!("{:?} len={}", s, s.len()); // -> Stack { items: [1, 200], pushes: 2 } len=2
println!("{:?}", s.pop()); // -> Some(200)
s.with(7).with(8); // chaining via &mut Self
println!("{:?}", s); // -> Stack { items: [1, 7, 8], pushes: 4 }
s.clear();

```

Three rules that follow

- **The binding must be `mut`.** `s.push(1)` is `Stack::push(&mut s, 1)`, and you cannot take `&mut s` from an immutable binding. (error E0596)
- **Exclusivity.** While the `&mut` borrow lives, nothing else may borrow the value — not even for reading. (errors E0502, E0499)
- **A borrow ends at its last use**, not at the end of the enclosing block (non-lexical lifetimes). That is why `s.pop()` above is accepted right after the `top_mut()` block: the mutable borrow is already dead.

Dereferencing: `*t` versus `t`

In `*t *= 100`, `t` is a `&mut i32`, so you dereference with `*` to reach the number. Method calls auto-dereference — `self.items.push(..)` works through a `&mut self` with no `*` at all — but a plain assignment through a reference needs it written out.

4. self — by value (consuming)

Take **self** when the method should **use the value up**: a conversion, or a builder step that threads the value onward. After the call the original binding is gone. Write **mut self** when you also want to reassign fields inside the method — that **mut** is about your local copy, and callers never see it in the signature's meaning.

Four consuming methods, including the builder pattern

```
#[derive(Debug)]
struct Config { host: String, port: u16, tls: bool }
#[derive(Debug)]
struct Server { addr: String }

impl Config {
    fn new(host: &str) -> Self { Self { host: host.into(), port: 80, tls: false } }

    // 1. consuming builder: take self, hand back Self
    fn port(mut self, p: u16) -> Self { self.port = p; self }
    fn tls(mut self, on: bool) -> Self { self.tls = on; self }

    // 2. terminal conversion: the Config is gone, a Server remains
    fn build(self) -> Server {
        Server { addr: format!("{}",
                                if self.tls { "https" } else { "http" },
                                self.host, self.port) }
    }

    // 3. choose between two owned values and return the winner
    fn or(self, fallback: Self) -> Self {
        if self.port != 0 { self } else { fallback }
    }

    // 4. destructure self into its parts, moving the String out
    fn into_parts(self) -> (String, u16) { (self.host, self.port) }
}

let server = Config::new("example.com").port(8443).tls(true).build();
// -> Server { addr: "https://example.com:8443" }
```

Builder: self or &mut self?

fn port(mut self) -> **Self** chains beautifully on a temporary (`Config::new(..).port(..).build()`) but forces a move each step. **fn port(&mut self)** -> **&mut Self** chains on a named **mut** variable and avoids the moves, but cannot end in a `build(self)`. Pick consuming for one-shot construction, **&mut** for long-lived, repeatedly-reconfigured objects.

Naming conventions that signal the receiver

Prefix	Receiver	Meaning	Example
as_	&self	cheap borrow-to-borrow view	String::as_str
to_	&self	expensive copy, original kept	str::to_string
into_	self	consumes, transfers ownership	String::into_bytes

5. No receiver — associated functions

A function in an `impl` block with no `self` parameter is an **associated function**. There is no instance, so you call it on the type with `::`, not with a dot. Constructors are the usual case, and this is where `Self` earns its keep as a return type.

```
impl Grid {
    fn new(w: usize, h: usize) -> Self { /* ... */ } // Grid::new(3, 4)
    fn square(n: usize) -> Self { Self::new(n, n) } // Grid::square(4)
}

let g = Grid::square(4); // dot notation would not compile: no receiver to pass
```

6. Self — the type

`Self` is an automatically available alias for the type the current block is about. It is usable anywhere a type name is, plus as a path prefix (`Self::`) to reach that type's own associated items. Four places it pays off:

(a) Constructors, struct literals and `Self::` paths

```
struct Grid { w: usize, h: usize, cells: Vec<u8> }

impl Grid {
    const MAX: usize = 1024;

    fn new(w: usize, h: usize) -> Self {
        Self { w, h, cells: vec![0; w * h] } // Self { .. } struct literal
    }
    fn square(n: usize) -> Self { Self::new(n, n) } // Self:: calls a fn
    fn biggest() -> Self { Self::square(Self::MAX.min(4)) } // Self:: reads a const
}
```

Rename `Grid` to `Board` and not one line of the body needs touching — that is the practical argument for preferring `Self` over repeating the type name.

(b) Generic impls — `Self` carries the parameters

```
struct Pair<T> { a: T, b: T }

impl<T: Clone + PartialOrd> Pair<T> {
    // inside this block, Self means Pair<T> -- parameters included
    fn new(a: T, b: T) -> Self { Self { a, b } }

    fn sorted(self) -> Self {
        if self.a <= self.b { self } else { Self { a: self.b, b: self.a } }
    }

    // Self works in parameter position too
    fn merge(&self, other: &Self) -> Self {
        Self { a: self.a.clone(), b: other.b.clone() }
    }
}

let p = Pair::new(9, 4).sorted(); // a=4 b=9
```

(c) Traits — `Self` is the implementor, whoever that turns out to be

In a trait definition the concrete type is not known yet, so `Self` is the only way to say “whatever type implements me”. Each `impl` substitutes its own:

```

trait Zeroed {
    fn zeroed() -> Self;           // no receiver; returns the implementing type
    fn doubled(self) -> Self;      // consumes and returns the implementing type
}

impl Zeroed for i32 {
    fn zeroed() -> Self { 0 }           // Self == i32
    fn doubled(self) -> Self { self * 2 }
}

impl Zeroed for String {
    fn zeroed() -> Self { String::new() } // Self == String
    fn doubled(self) -> Self { format!("{0}{0}", self) }
}

i32::zeroed();           // -> 0
21.doubled();            // -> 42
String::from("ab").doubled(); // -> "abab"

```

This is exactly how the standard library declares `Default::default() -> Self`, `Clone::clone(&self) -> Self` and `Add::add(self, rhs) -> Self::Output`.

(d) Enums — `Self::` reaches your own variants

```

#[derive(Debug)]
enum Dir { N, S, E, W }

impl Dir {
    fn flip(self) -> Self {
        match self {
            Self::N => Self::S, // no need to repeat `Dir::`
            Self::S => Self::N,
            Self::E => Self::W,
            Self::W => Self::E,
        }
    }
}

Dir::N.flip(); // -> S

```

Where `Self` is and is not available

Context	<code>Self</code> available?
inside <code>impl</code> <code>Type</code> { .. }	yes — means <code>Type</code> (with its generic arguments)
inside <code>trait</code> <code>T</code> { .. }	yes — means the future implementor
inside a <code>struct/enum</code> definition body	no — write the type name
a free function outside any <code>impl</code>	no
as a path prefix: <code>Self::CONST</code> , <code>Self::func()</code> , <code>Self::Variant</code>	yes

7. The four errors you will actually hit

These are real messages from `rustc 1.98.1`, each produced by a minimal program.

E0596 — binding is not mut

```
let c = Counter { n: 0 }; // needs: let mut c = ...
c.bump();                // bump(&mut self)
```

```
error[E0596]: cannot borrow `c` as mutable, as it is not declared as mutable
```

Fix: `let mut c = ...`. Remember `c.bump()` means `Counter::bump(&mut c)`.

E0502 — shared borrow while a mutable one is live

```
let t = s.top_mut(); // &mut borrow of s starts
println!("{}", s.len()); // &self borrow of s -- rejected
println!("{}", t); // ...because the &mut borrow is still used here
```

```
error[E0502]: cannot borrow `s` as immutable because it is also borrowed as mutable
```

Fix: do not keep both alive. Read first, then mutate — each borrow dying at its last use:

```
let len = s.len(); // borrow starts and ends here
if let Some(t) = s.top_mut() { *t += 1; } // and here
println!("{}", len);
```

E0499 — two mutable borrows at once

```
let x = s.a_mut(); // -> &mut i32 borrowed from s
let y = s.b_mut(); // second &mut borrow of s -- rejected
*x += *y;
```

```
error[E0499]: cannot borrow `s` as mutable more than once at a time
```

The compiler cannot see through the method calls to know the two fields are disjoint. **Fix:** borrow the fields directly (`let (x, y) = (&mut s.a, &mut s.b);`), which the borrow checker does split per-field, or return both from one method, or copy one value out first.

E0382 — use after move

```
let a = p.consume(); // consume(self) moves p away
let b = p.consume(); // p is gone -- rejected
```

```
error[E0382]: use of moved value: `p`
```

Fix: change the signature to `&self` if the method does not really need ownership, or clone before consuming, or restructure so the value is consumed exactly once.

8. Choosing a receiver

You want to...	Use	Note
read fields, compute, format, compare	<code>&self</code>	start here — the default
assign to <code>self.field</code> , push, clear, sort	<code>&mut self</code>	binding must be <code>mut</code>

You want to...	Use	Note
convert the value into something else	<code>self</code>	caller loses the value
chain builder steps on a temporary	<code>mut self -> Self</code>	one move per step
chain on a long-lived object	<code>&mut self -> &mut Self</code>	no moves
construct, with no instance yet	no receiver	<code>Type::new()</code> -> <code>Self</code>

Rules of thumb

- Write `&self` first. Promote to `&mut self` only when the compiler asks, i.e. when you actually assign to something reachable from `self`.
- Reach for `self` only when the value genuinely should be unusable afterwards — `build()`, `into_parts()`, `into_iter()`.
- Prefer `Self` to repeating the type name in return types, struct literals and `Self::` paths: it survives renames and reads better in generic impls.
- If a `&mut self` method fights the borrow checker, the usual cure is to narrow the borrow (take a `Copy` value out first) or to split the struct so the two things you need to touch live in different fields.

Appendix — verification

Every snippet above is an extract from one program compiled with `rustc --edition 2024`. Its actual output was:

```
[&self] Bora (32) adult=true name=Bora same_age=true
[&self] reused after borrows: Bora (32)
[&mut self] Stack { items: [1, 200], pushes: 2 } len=2
[&mut self] popped Some(200)
[&mut self] chained Stack { items: [1, 7, 8], pushes: 4 }
[&mut self] cleared Stack { items: [], pushes: 4 }
[self] Server { addr: "https://example.com:8443" }
[self] parts host=a port=80
[Self] grid 4x4 cells=16
[Self] pair a=4 b=9
[Self] trait i32: 0 42
[Self] trait String: "" "abab"
[Self] enum N -> S
[desugar] a=10
[desugar] after b: 11
[desugar] boxed d=42
[desugar] c=11
```

The four error listings come from four separate minimal files, each compiled to confirm the exact message and code.